Trabajo Práctico Integrador — Parcial II

Food Store: capa de datos completa (PostgreSQL, MongoDB y SQLite) · Base de Datos II

Materia: Base de Datos II · **Profesor:** Neira · **Grupo:** 4
Integrantes: Belén Calvo, Elías Tello, Hernán González, Bruno Fiouchetta
Fecha: noviembre de 2026
Repositorios: base-datos-2 y BaseDatos2-MongoDB, ambos con la etiqueta `tpi-parcial2`

Organización de este informe según la consigna: los puntos 1 (qué se implementó), 3 (resultados) y 4 (decisiones de diseño) se tratan por unidad, dentro de la sección 1; el punto 2 (cómo se probó) va en la sección 2; y el punto 5 (herramientas de IA) va en la sección 3.

1. Qué se implementó, resultados y decisiones por unidad

Unidad 4 — Normalización y redundancia

Qué se implementó: Se analizó formalmente la relación `control_lote_almacen(lote_id, deposito_id, responsable_control_id)` frente a sus dependencias funcionales reales F_1: {lote_id, deposito_id} → responsable_control_id y F_2: responsable_control_id → deposito_id, determinando que cumple 3FN (todos sus atributos son primos al integrar las claves candidatas {lote_id, deposito_id} y {lote_id, responsable_control_id}) pero **viola FNBC** porque el determinante de F_2 no es superclave (*food-store/specs/spec_fnbc_control_lote.md*, *docs/informe_tp_u4.md*). Bajo especificación previa en Kiro (*food-store/specs/spec_fnbc_descomposicion.md*), se implementó la descomposición sin pérdida en dos tablas en FNBC — `responsable_deposito(responsable_control_id PK, deposito_id)` (R_1) y `control_lote(lote_id, responsable_control_id)` (R_2, con FK a R_1)—, la vista de compatibilidad `v_control_lote_almacen` con `NATURAL JOIN` (*food-store/tp_fnbc_control_lote.sql*) y el plan y script de reversión transaccional hacia `control_lote_almacen` respetando el orden de dependencias y claves foráneas sin `CASCADE` (*food-store/tp_fnbc_reversion.sql*). En la Parte 2 se implementó una desnormalización controlada para la consulta del top de categorías del día mediante la tabla `resumen_ventas_categoria_dia(fecha, id_categoria, total_vendido, cantidad_lineas)`, mantenida por un único dueño (`fn_resumen_ventas_aplicar_linea`) invocado desde `trg_resumen_ventas_detalle` (`AFTER INSERT/UPDATE/DELETE ON detalle_pedido`) y `trg_resumen_ventas_pedido_borrado` (`BEFORE DELETE ON pedido`), junto con su auditoría y su script de reversión (*food-store/specs/spec_top_categorias_desnormalizacion.md*, *food-store/tp_desnormalizacion_top_categorias.sql*, *food-store/tp_desnormalizacion_reversion.sql*).

Cómo se probó y resultado: Sobre la copia `bd2_tp4u4`, las anomalías de inserción, borrado y actualización de `control_lote_almacen` se demostraron aislando cada prueba con `SAVEPOINT` y `ROLLBACK TO SAVEPOINT` (*food-store/tp_fnbc_salida_rollback.txt*). La migración a FNBC pobló 2 filas en `responsable_deposito` y 3 en `control_lote`, y la verificación con `EXCEPT` en ambas direcciones entre `control_lote_almacen` y `v_control_lote_almacen` devolvió **0 filas** (*food-store/tp_fnbc_salida_rollback.txt*). El ciclo completo de reversión (`migrar → verificar → revertir → verificar`), probado primero con `ROLLBACK` y luego con `COMMIT` (mediante `-v commit=1` sin editar el script), reconstruyó las 3 tuplas desde `control_lote NATURAL JOIN responsable_deposito`, eliminó `v_control_lote_almacen`, `control_lote` y `responsable_deposito` en orden de FK y devolvió **0 filas** en ambos `EXCEPT` contra la foto previa (*food-store/tp_fnbc_reversion_salida.txt*). En la desnormalización (220.005 pedidos y 539.286 detalles), la mediana de 5 corridas con `EXPLAIN (ANALYZE, BUFFERS)` bajó de **78,077 ms** (7.770 buffers, `Parallel Seq Scan on detalle_pedido`) a **0,037 ms** (8 buffers, `Bitmap Index Scan` en la PK del resumen), una mejora de ~2.100 veces a cambio de +0,048 ms por línea insertada (de 22,653 ms a 46,591 ms en 500 `INSERT`, y +39 % de WAL) (*food-store/medicion_antes_salida_bruno.txt*, *food-store/medicion_despues_salida_bruno.txt*, *food-store/medicion_escritura_salida_bruno.txt*). La auditoría dio **0 filas** tras operar el trigger y detectó las 3 desincronizaciones forzadas en `SAVEPOINT` (*food-store/auditoria_salida_bruno.txt*).

Decisión de diseño: Se aceptó la descomposición a FNBC documentando su *trade-off*: impone estructuralmente F_2 por PK de `responsable_deposito`, aunque deja de preservar F_1 en una sola tabla, motivo por el cual se especificó y probó el plan de reversión (*food-store/specs/spec_fnbc_descomposicion.md*). Para el ranking diario se eligió la tabla resumen mantenida por trigger y se descartó la vista materializada porque el panel exige datos en tiempo real sin ventana de desfase; además se agregó `cantidad_lineas` para borrar la fila solo cuando queda sin líneas (dado que `precio_unitario` admite 0) y se descartó indexar `total_vendido` para no penalizar cada venta ni perder actualizaciones HOT por ordenar apenas 5 categorías (*food-store/specs/spec_top_categorias_desnormalizacion.md*).

Unidad 5 — Administración y seguridad

Qué se implementó: Bajo el principio de mínimo privilegio y especificación en Kiro (*docs/kiro_roles_spec/requirements.md*, *docs/kiro_roles_spec/design.md*), se implementó en `sql/roles.sql` un esquema separando 4 roles de grupo sin login (`rol_app_lectura`, `rol_app_escritura`, `rol_soporte`, `rol_reportes` con `NOLOGIN`) y 3 roles de login sin superusuario (`app_web`, `rol_reportes_login` y `admin_datos`, dueño del esquema `public` y de sus objetos). La autenticación (`fn_autenticar`) y el reseteo de claves (`fn_resetear_password`) se encapsularon en `sql/funciones_auth.sql` con `pgcrypto` (`crypt/gen_salt('bf')`), `SECURITY DEFINER` y `SET search_path = public, pg_temp`, al igual que el procedimiento transaccional `registrar_pedido`, mientras que para reportes se creó `mv_ventas_categoria_mes` (*sql/mv_ventas_categoria_mes.sql*). En la Parte B se activó la auditoría nativa (`logging_collector = on`, `log_connections = on`, `log_disconnections = on`, `log_statement = 'mod'`, `log_line_prefix = '%m [%p] %q%u@%d %r %a '`) (*sql/auditoria_test.sql*, *docs/informe_auditoria.md*). En la Parte C se ejecutó el simulacro de incidente y su anonimizador (*sql/simulacro_incidente.sql*, *sql/anonimizar_log.sh*, *docs/informe_incidente.md*), y en la Parte D la tabla anonimizada `usuario_anon` (*sql/usuario_anon.sql*, *docs/verificacion_equivalencia.md*). Finalmente, se documentó la decisión humana y comprobación en motor de los

hallazgos `H1–H5`, `F1–F4`, `R1–R3` y `M1–M4` de la auditoría de roles asistida por IA (*docs/auditoria_roles_ia.md*, *docs/decisiones_auditoria_roles.md*).

Cómo se probó y resultado: Sobre `bd2_tp5u5`, `sql/verificacion_roles.sql` comprobó dentro de `BEGIN … ROLLBACK` con `SET ROLE` que: `app_web` autentica con `fn_autenticar` y registra ventas con `CALL registrar_pedido`, pero recibe `permiso denegado` al intentar `SELECT` sobre `usuario`, `INSERT`/`UPDATE` directo sobre `pedido`, `CREATE TABLE` en `public` o `GRANT admin_datos TO rol_soporte`; `rol_soporte` lee `id, nombre, apellido, mail, rol` de `usuario` y resetea claves de cuentas `USUARIO` (≥ 8 caracteres), pero falla al leer `contrasena`, `celular` o `eliminado` y al intentar resetear al usuario 1 (`ADMIN`); y `rol_reportes_login` lee las 125 filas de `mv_ventas_categoria_mes` pero falla sobre `cliente`, `vista_usuario_reportes`, `fn_autenticar` y `REFRESH MATERIALIZED VIEW` (*docs/verificacion_roles.txt*). En el simulacro de acceso indebido (304 usuarios), los 3 intentos fallidos, el login válido, el `SELECT * FROM usuario` denegado y el `GRANT`/`SET ROLE admin_datos` rechazados quedaron trazados en `docs/log_simulacro_anonimizado.txt` (39 líneas sin datos reales). En `usuario_anon`, la consulta analítica por rol y mes de alta sobre 303 usuarios sintéticos dio **0 filas** de diferencia con `EXCEPT` en ambas direcciones frente a `usuario` (*docs/verificacion_equivalencia.md*).

Decisión de diseño: De la auditoría de permisos asistida por IA (*docs/decisiones_auditoria_roles.md*, 16 hallazgos analizados) se **aceptaron** `H1` (revocar `INSERT, UPDATE` directo sobre `pedido` y `detalle_pedido` en `rol_app_escritura` para obligar el paso por `registrar_pedido`) y `H4` (impedir en `fn_resetear_password` el reseteo de cuentas `ADMIN` y exigir 8 caracteres), se **corrigieron** `R1` y `R2`, y se **descartaron con fundamento técnico o documentaron como riesgo aceptado** crear un esquema `reportes` aparte (`H2`), tocar `bd2_tp3` o scripts de TPs previos (`H3`, con `CONNECT` vía `PUBLIC` comprobado por `has_database_privilege`, `F1`, `R2`), mantener `CREATEDB` y `CREATEROLE` en `admin_datos` para operar sin superusuario, agregar el rol `soporte_login`, un rol de refresco o la columna `eliminado` (`F2`, `F3`, por violar mínimo privilegio) y reescribir `roles.sql` con loops sobre catálogos (`R1`, `M1–M4`, porque dificulta la auditoría línea por línea y arriesga revocar `EXECUTE` sobre `pgcrypto`). En el incidente (*docs/informe_incidente.md*) se descartó la sugerencia de la IA de matar procesos ya cerrados y rotar la clave de `app_web` (que dejaría caída toda la app), decidiendo verificar y contener únicamente la cuenta de aplicación `usuario_001`.

Unidad 6 — Mantenimiento y simulacro de desastre

Qué se implementó: Un plan de mantenimiento para la base `foodstore`, especificado en *mantenimiento/SPEC_U6.md*, con cuatro tareas. **Respaldo diario:** `backup_foodstore.sh` hace `pg_dump -Fc`, verifica el archivo con `pg_restore -l`, guarda su SHA256 y una huella de referencia de 11 claves (*mantenimiento/conteo_referencia.sql*: filas de las 6 tablas, total facturado y cantidad de tablas, vistas, restricciones e índices), y respalda los roles aparte con `pg_dumpall --globals-only --no-role-passwords`. Lo programa la tarea de Windows `FoodStore_Backup_Diario` (21:00, `StartWhenAvailable`, sin contraseña guardada; *mantenimiento/programar_respaldo.ps1*). Aplica una **retención** de 7 días, sin bajar nunca de 3 juegos y solo después de un PASS. **Chequeo de salud** con *mantenimiento/health_check.sql*. **Mantenimiento** con *mantenimiento/maintenance.sql*: `ANALYZE`/`VACUUM` solo si las métricas superan los umbrales de autovacuum, nunca `VACUUM FULL`, y sale con código 3 si la base no es la permitida. **Restauración de prueba** con *mantenimiento/restore_test.sh* en la base descartable `foodstore_restore_test`. Sobre ese plan se especificó y ejecutó un **simulacro de desastre contra reloj** (*mantenimiento/SPEC_simulacro_desastre.md*, *mantenimiento/simulacro_desastre.sh*): en la copia `foodstore_desastre` se registran 3 pedidos después del último respaldo de la tarea y se ejecuta `DELETE FROM detalle_pedido;` sin `WHERE`.

Cómo se probó y resultado: La tarea registró respaldos `PASS` con `origen=programada`, uno de ellos disparado por el horario real (*mantenimiento/logs/backup_foodstore.log*, *mantenimiento/evidencias/tarea_programada_tercera_corrida.txt*). La retención se probó en una carpeta temporal: borró 3 juegos de más de 7 días y dejó 3, y con una configuración inválida o una base inexistente no borró nada (*mantenimiento/evidencias/retencion_pruebas.txt*). El respaldo de roles trae 9 roles y 0 hashes de contraseña (*mantenimiento/evidencias/globals_pruebas.txt*). La restauración de prueba dio huella 11/11 igual al respaldo y los 8 indicadores de integridad en 0, con un RTO de 4,22 s (*mantenimiento/evidencias/restore_test_20261108_034017.txt*). En el simulacro de desastre, primero en ensayo dentro de `BEGIN … ROLLBACK` y después en serio, el incidente se detectó 1 s después del `DELETE` y la base quedó restaurada y validada **6,16 s después de la detección** (meta ≤ 15 min). El RPO observado fue de 514 s (meta ≤ 24 h) y se perdieron exactamente los **3 pedidos y 6 líneas** registrados después del respaldo. La huella de `foodstore` fue igual antes y después, así que el simulacro no la tocó (*mantenimiento/evidencias/simulacro_desastre_20261108_235715.txt*).

Resumen del informe post-incidente (*mantenimiento/informe_post_incidente.md*, sin culpables): la causa raíz fue una sentencia destructiva ejecutada fuera de una transacción y con una cuenta administrativa; ningún rol de la aplicación tiene `DELETE` sobre `detalle_pedido` (*sql/roles.sql*). El simulacro mostró una falla de detección que no estaba prevista: el autovacuum limpió las tuplas muertas 73 s después del `DELETE`, y pasado ese minuto `health_check.sql` no muestra el problema. Se proponen como acciones un control de pedidos sin líneas y, para bajar el RPO de hasta 24 h a minutos, el archivado continuo de WAL (recuperación a un punto en el tiempo).

Decisión de diseño: Se validó contra la **huella guardada junto al respaldo** y no contra la base viva: así `restore_test.sh` nunca consulta `foodstore`, y la comparación vale aunque la base cambie después del respaldo. El simulacro **restaura aparte** y deja la base dañada para medir qué se perdió, en vez de restaurar sobre ella. Toda base que se borra es descartable y pide **autorización escrita**, que queda en el log. Los permisos sobre las tablas no se recuperan en la restauración (`--no-owner --no-privileges`): es una limitación declarada en *mantenimiento/SPEC_U6.md*.

Unidad 7 — NoSQL y elección de tecnología

Qué se implementó: Se migraron 8 pedidos y 30 líneas desde `pedido`, `detalle_pedido`, `cliente` y `producto` de PostgreSQL a la colección `pedidos` de MongoDB (*modelo_documental.md*). Cada documento conserva `_id` igual a `id_pedido`, embebe a `cliente` y guarda el arreglo `lineas[]` (*modelo_documental.md*). El `total` se calcula en PostgreSQL como suma de `cantidad` por `precio_unitario` y se guarda como `Decimal128`; el subtotal de cada línea no se guarda (*modelo_documental.md*).

Cómo se probó y resultado: La auditoría de totales devolvió 0 pedidos con total distinto de la suma de líneas (*evidencias/salida_migracion_y_verificacion.txt*). Las consultas por forma de pago, por producto y por total mayor que 10000 coincidieron en IDs y cantidades entre ambos motores: 3, 3 y 3 pedidos (*consigna5_tabla_comparativa.md*). La prueba del índice único propuesto por la IA aceptó un duplicado dentro del mismo pedido y rechazó con código 11000 un pedido legítimo con el mismo producto (*evidencias/salida_prueba_indice_unico_ia.txt*).

Decisión de diseño: Se descartó el índice único y la unicidad se impuso con validador `$jsonSchema` y `$setUnion`, con rechazo de código 121 ante producto repetido (*evidencias/salida_migracion_y_verificacion.txt*). Se descartó el punto flotante en favor de `Decimal128` (*consigna6_ia_contraste.md*). PostgreSQL queda como sistema de registro y MongoDB como lectura de pedidos completos en una sola lectura (*consigna6_ia_contraste.md*).

Unidad 8 — Móvil e integración

Qué se implementó: La base de la tablet se especificó en Kiro con 6 requisitos en notación EARS, casos C1 a C7 y la regla de que la tablet propone y el servidor dispone (*.kiro/specs/tp8-tablet/requirements.md*). El esquema y las pruebas los generó OpenCode (agente Build) a partir de las tareas de esa especificación, con revisión completa antes de commitear y un descarte documentado (*.kiro/specs/tp8-tablet/tasks.md*, *food-store/docs/DUIA.md*). El diseño define 6 tablas `STRICT` en SQLite, con precios en centavos enteros y claves foráneas activas (*.kiro/specs/tp8-tablet/design.md*). La tablet guarda una copia fechada del catálogo, arma borradores sin conexión y encola solicitudes para el semáforo (*food-store/docs/2c_casos_y_3a_mapa.md*).

Cómo se probó y resultado: Las pruebas de SQLite verificaron borrador con `COMMIT` de 1 fila por tabla, subtotal de 300000 centavos, `ROLLBACK` sin cambios, catálogo sin servidor con 1 producto y los 2 errores esperados de `CHECK` y clave foránea (*food-store/evidencias/salida_pruebas_sqlite.txt*). En los casos, C2 pasó de `CONFLICTO` por cambio de precio de 140000 a 160000 centavos a `SIMULADO_APROBADO`, C3 quedó `RECHAZADO` porque pide 2 y hay 1, C4 quedó `RECHAZADO` por baja y C5 quedó `SIMULADO_APROBADO` (*food-store/evidencias/salida_casos_c2_c6_sqlite.txt*). C6 mantuvo 5 evaluaciones antes y después, con 1 solo error esperado de unicidad y reenvío de 0 filas (*food-store/evidencias/salida_casos_c2_c6_sqlite.txt*). La instantánea tuvo 50011 filas y 10049 lápidas (*food-store/evidencias/salida_casos_c2_c6_sqlite.txt*). PostgreSQL quedó con 200005 pedidos antes y después de simular (*food-store/docs/2c_casos_y_3a_mapa.md*).

Decisión de diseño: Se descartó convertir el precio de la vista a texto y se mantuvo el valor numérico para ordenar y filtrar, y se descartó `UNIQUE` total sobre `id_local` en favor del índice parcial por estado pendiente, para permitir el reenvío tras un conflicto (*food-store/docs/3c_code_review_ia.md*). El code review final se hizo con OpenCode en solo lectura sobre los scripts de U1 a U8, con decisión documentada por hallazgo (*food-store/docs/3c_code_review_ia.md*).

2. Cómo se probó (transversal)

Todas las pruebas del TPI siguieron las mismas reglas. Se trabajó **sobre copias**, nunca sobre `bd2_tp3` ni `foodstore`: `bd2_tp4u4` (U4), `bd2_tp5u5` (U5), `foodstore_restore_test` y `foodstore_desastre` (U6) y bases SQLite propias (U8). Cada script que cambia datos empieza con una **guarda de base** (`current_database()` con `RAISE EXCEPTION`) y corre **primero en** `BEGIN … ROLLBACK`: en U4 la migración y la reversión se probaron con `ROLLBACK` y después con `COMMIT` (`-v commit=1`, sin editar el script); en U5 cada permiso y cada rechazo se verificó con `SET ROLE` dentro de una transacción revertida; en U6 el simulacro tiene un modo ensayo. La **equivalencia** se comprobó con `EXCEPT` en las dos direcciones (0 filas en la descomposición FNBC y en `usuario_anon`) y, entre motores, comparando cantidades e identificadores de pedidos en PostgreSQL y MongoDB (U7). El **rendimiento** se midió con `EXPLAIN (ANALYZE, BUFFERS)`, tomando la mediana de varias corridas (U4). La **recuperación** se validó con la huella de 11 claves, el SHA256 del respaldo y 8 indicadores de integridad, y se midieron RTO y RPO en dos simulacros (U6). Hubo dos **simulacros de incidente**: acceso indebido reconstruido desde el log anonimizado (U5) y desastre contra reloj (U6). La **sincronización móvil** se probó con los casos de conflicto C1 a C7 en SQLite y en el servidor (U8). Cada prueba deja su salida en un `.txt` versionado. Las propuestas de la IA se trataron como hipótesis: cada una se confirmó o se descartó en el motor, citando el error literal o la evidencia. Todo se reproduce desde el README de cada repositorio, y *docs/tpi_parcial2/matriz_objetivos.md* lleva de cada objetivo a sus evidencias.

3. Herramientas de IA y code review

Qué se implementó: A lo largo de las Unidades 4 a 8 y del cierre integrador se aplicó la regla de la cátedra "*se delega la escritura, nunca la decisión*", articulando tres roles de herramientas y registrando toda intervención y apartamiento en la DUIA consolidada (*docs/tpi_parcial2/DUIA_tpi.md*) y en la bitácora (*docs/tpi_parcial2/bitacora_ia_tpi.md*). Con **Kiro** se especificaron previamente los requisitos verificables (notación EARS en U5 y U8), las invariantes, los planes de reversión y las tareas: descomposición a FNBC, plan de reversión y desnormalización del top de categorías en U4 (*food-store/specs/spec_*.md*); modelo de mínimo privilegio en U5 (*docs/kiro_roles_spec/*.md*); plan de mantenimiento, programación y retención en U6 (*mantenimiento/SPEC_U6.md*; la spec del simulacro de desastre, *mantenimiento/SPEC_simulacro_desastre.md*, se redactó con Claude Code como apartamiento declarado); y base móvil offline en SQLite en U8 (*.kiro/specs/tp8-tablet/{requirements,design,tasks}.md*). Con **OpenCode (agente Build)** y asistentes de entorno se generaron los scripts `.sql`, `.sh`, `.ps1` y `.js` a partir de esas especificaciones, sin enviar jamás contraseñas ni datos personales reales a la IA (*sql/anonimizar_log.sh*, *sql/usuario_anon.sql*, `pg_dumpall --no-role-passwords`). Finalmente, con **OpenCode (agente Plan, modo de solo lectura)** se ejecutó el code review completo de los **56 scripts de la cursada más los 3 scripts nuevos del TPI** en ambos repositorios (*food-store/docs/inventario_scripts_u1_u8.md*, con **0 scripts sin revisar**), documentando los informes por unidad (*docs/tpi_parcial2/review/review_u1.md* a *review_u8.md*, *mantenimiento/revision_opencode.md*, *mantenimiento/revision_opencode_v2.md* y *docs/auditoria_roles_ia.md*) y la tabla consolidada de 18 hallazgos (*food-store/docs/3c_code_review_ia.md*).

Cómo se probó y resultado: Ningún script ni propuesta de la IA se ejecutó sobre `bd2_tp3` ni sobre `foodstore` en operaciones destructivas: cada hallazgo o sugerencia del code review se contrastó contra la especificación y se comprobó en el motor

correspondiente (`psql` 17.11, `mongosh` 2.14.1 o `sqlite3` 3.49.1) dentro de transacciones reversibles (`BEGIN … ROLLBACK`) sobre las copias de trabajo (`bd2_trabajo`, `bd2_tp4u4`, `bd2_tp5u5`, `foodstore_restore_test`, `bd2_tp8` y `foodstore_movil.db`). De los 59 scripts auditados en el inventario final (*food-store/docs/inventario_scripts_u1_u8.md*), 21 quedaron verificados sin observaciones tras comprobar sus guardas y salidas, y 38 quedaron vinculados a los **18 hallazgos** de *food-store/docs/3c_code_review_ia.md* (además de las 16 decisiones de *docs/decisiones_auditoria_roles.md* en U5 y las 13 de *mantenimiento/decisiones_revision.md* en U6), verificando cada corrección o rechazo con el mensaje de error literal del motor, el plan de `EXPLAIN (ANALYZE, BUFFERS)` o la evidencia `.txt` versionada.

Decisión de diseño: Se exigió al menos una propuesta descartada con fundamento técnico y prueba en el motor por cada bloque e integrante (*docs/tpi_parcial2/bitacora_ia_tpi.md*):

Unidad	Qué se aceptó / corrigió tras comprobar en el motor	Qué se descartó y por qué (comprobado en `psql`, `mongosh` o `sqlite3`)
U1 a U3	**Corregido (H1):** `registrar_pedido` fijó `SECURITY DEFINER` y `search_path = public, pg_temp` (*sql/roles.sql*). **Aceptado:** índice cubriente `idx_producto_categoria_precio` con `INCLUDE` (baja C1 de 10,5 ms a 3,1 ms con `Index Only Scan`, *food-store/indices.sql*).	**Descartado (H12–H14):** 1) `CHECK (cantidad <= (SELECT stock …))` en `detalle_pedido`: falla con `ERROR: no se pueden usar subconsultas en una restricción «check»` (*review_u1.md*). 2) Quitar `+ s.i * 0` en `ORDER BY random()` de `carga_masiva.sql`: `EXPLAIN` demostró que pasa a `InitPlan` (`loops=1`) y repite 1 sola categoría y 1 solo cliente (*review_u2.md*). 3) Índice `DATE_TRUNC('month', fecha)` sobre `TIMESTAMPTZ`: falla con `ERROR: las funciones utilizadas en expresiones de índice deben estar marcadas IMMUTABLE` (*review_u3.md*).
U4	**Corregido (H3):** trigger `BEFORE DELETE ON pedido` (`trg_resumen_ventas_pedido_borrado`) para descontar antes del `ON DELETE CASCADE` (*food-store/tp_desnormalizacion_top_categorias.sql*), y `TRUNCATE control_lote_almacen` antes de reconstruir desde $R_1 \bowtie R_2$ (*food-store/tp_fnbc_reversion.sql*).	**Descartado (H15):** 1) Filtrar con `WHERE ped.fecha::date = CURRENT_DATE`: `EXPLAIN` comprobó que el cast fuerza `Parallel Seq Scan` (costo `3882.76`) e impide el `Index Only Scan` del rango `[CURRENT_DATE, CURRENT_DATE + 1)` (costo `8.45`, *review_u4.md*). 2) Usar `LEFT JOIN` en `auditoria_top_categorias.sql` y `DROP TABLE … CASCADE` en `tp_fnbc_reversion.sql` (oculta el orden de FK: vista → R_2 → R_1).
U5	**Corregido (H1, H4, H5, H16):** revocación de `INSERT, UPDATE` directo en `pedido` / `detalle_pedido` a `rol_app_escritura`; bloqueo de reseteo a `ADMIN` y claves < 8 caracteres en `fn_resetear_password`; lectura del log en `WIN1252` (*sql/auditoria_test.sql*); y quita de `eliminado` en la subconsulta de `rol_soporte` (*sql/verificacion_roles.sql*).	**Descartado (10 hallazgos en `decisiones_auditoria_roles.md` + H5, H16):** dejar `log_statement = 'all'` global; crear esquema `reportes` (`H2`); dar `SELECT(eliminado)` a `rol_soporte` (`F3`, viola mínimo privilegio); loops dinámicos sobre `pg_proc` (`R1`, arriesga revocar `pgcrypto`); rotar clave de `app_web` en el incidente; y en `anonimizar_log.sh` usar `'***'` fijo (impide distinguir fuerza bruta de reintentos; se usó `CLAVE_REDACTADA_N`) o regex de mail sin punto que pisa `app_web@bd2_tp5u5` (*review_u5.md*).
U6	**Corregido (H6, H7):** inclusión de `usuario` en `maintenance.sql` (`reltuples = -1`), evaluación condicional por umbral y guarda con `RAISE EXCEPTION` (código 3); huella `.conteo` de 11 claves, `pg_dumpall --globals-only --no-role-passwords` con 4 guardas, retención de 7 días (mín. 3 juegos tras `PASS`) y tarea diaria en `schtasks` (*mantenimiento/decisiones_revision.md*).	**Descartado (H8 y bitácora Bloque A):** 1) Marcar como defecto `n_live_tup = 0` en `health_check.sql` tras `createdb -T bd2_tp3`: ese `0` fue la señal real que justificó correr `maintenance.sql`. 2) Probar la retención sobre `backups/` real (se probó con `BACKUP_DIR` temporal, *retencion_pruebas.txt*) y ejecutar `quitar_respaldo.ps1` al final del README.
U7	**Corregido (H9):** importes como `Decimal128` (`NumberDecimal`) en lugar de `double` y unicidad de productos por pedido mediante `$jsonSchema` con `$expr` y `$setUnion` (rechazo código `121`, *evidencias/salida_migracion_y_verificacion.txt*).	**Descartado (H9, H17):** 1) Índice único multikey `createIndex({ "lineas.id_producto": 1 }, { unique: true })`: en `mongosh` (*prueba_indice_unico_ia.js*) aceptó un pedido con `id_producto: 1` repetido adentro y bloqueó con `E11000` un segundo pedido de otro cliente. 2) Guardar `'subtotal'` en `lineas[]` y filtrar `WHERE eliminado = FALSE` en `consultas_postgresql.sql` (`0 filas` con esa columna en `information_schema.columns`, *review_u7.md*).
U8	**Corregido (H11):** lápidas `NOT p.activo OR NOT c.activo` en `10_snapshot_catalogo.sql` (10.048 filas por `Congelados`), guarda con `DO $$ … RAISE EXCEPTION`, tabla `evaluacion_sync` e idempotencia de C6 (*food-store/docs/bitacora_ia.md*).	**Descartado (H10, H18):** 1) Formatear `precio` con `printf` como `TEXT` en `v_catalogo_disponible` (rompe `ORDER BY precio` numérico) y `UNIQUE(id_local)` total en `cola_sync` (bloquea el reenvío de C2; se usó índice parcial `WHERE estado = 'PENDIENTE'`). 2) Hacer `DELETE FROM producto_cache` en `importar_catalogo.sql` (falla con `FOREIGN KEY constraint failed` cuando hay `item_borrador`) o quitar `WHERE true` antes de `ON CONFLICT` (falla con `near "DO": syntax error`, *review_u8.md*).

Índice de objetivos → evidencia

Objetivo	Evidencia principal
1. U4 — Refactor a FNBC validada contra dependencias funcionales	R: `food-store/specs/spec_fnbc_control_lote.md`, R: `food-store/tp_fnbc_control_lote.sql`
2. U4 — Migración sin pérdida, en transacciones reversibles, especificada en Kiro, con plan de reversión	R: `food-store/specs/spec_fnbc_descomposicion.md`, R: `food-store/tp_fnbc_control_lote.sql`
3. U4 — Desnormalización controlada documentada	R: `food-store/specs/spec_top_categorias_desnormalizacion.md`, R: `food-store/tp_desnormalizacion_top_categorias.sql`
4. U5 — Usuarios, roles y permisos (mínimo privilegio)	R: `sql/roles.sql`, R: `sql/verificacion_roles.sql`
5. U5 — Auditoría activada y simulacro de acceso indebido reconstruido del log, con contención	R: `sql/auditoria_test.sql`, R: `sql/simulacro_incidente.sql`
6. U5 — Auditoría de permisos asistida por IA con decisión humana documentada	R: `docs/auditoria_roles_ia.md`, R: `docs/informe_auditoria.md`

Objetivo	Evidencia principal
7. U6 — Plan de mantenimiento completo (respaldo programado, restauración probada, tareas rutinarias)	R: `mantenimiento/SPEC_U6.md`, R: `mantenimiento/backup_foodstore.sh`
8. U6 — Simulacro de desastre contra reloj con informe post-incidente	R: `mantenimiento/simulacro_desastre.sh`, R: `mantenimiento/informe_post_incidente.md`
9. U7 — Modelado documental con consultas verificadas	M: `modelo_documental.md`, M: `consultas_mongodb.js`
10. U7 — Comparación documental vs relacional con justificación	M: `consigna5_tabla_comparativa.md`, M: `consigna6_ia_contraste.md`
11. U8 — SQLite especificado en Kiro y generado con OpenCode	M: `.kiro/specs/tp8-tablet/requirements.md`, M: `food-store/movil/schema_sqlite.sql`
12. U8 — Sincronización simulada con casos de conflicto probados	M: `food-store/movil/semaforo.sql`, M: `food-store/movil/casos_sqlite.sql`
13. U8 — Code review final asistido por IA de todos los scripts, con decisión humana por hallazgo	M: `food-store/docs/3c_code_review_ia.md`, R: `docs/tpi_parcial2/review/review_u1.md` a `review_u8.md`
14. Capa de datos completa lista para la defensa	R: `README.md`, R: `docs/tpi_parcial2/matriz_objetivos.md`